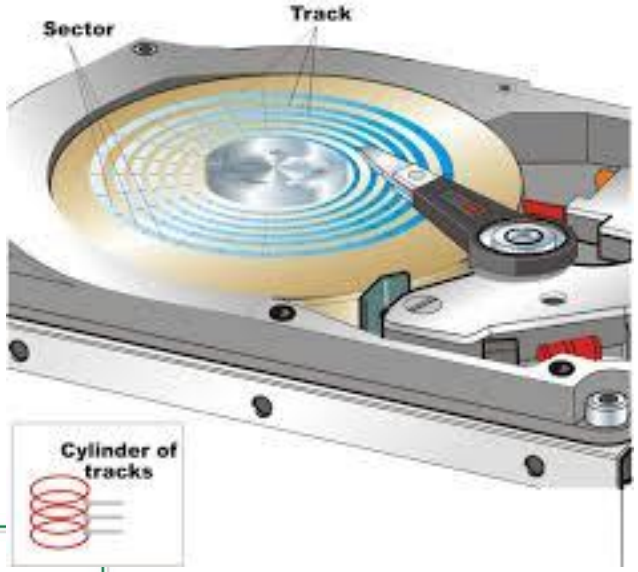
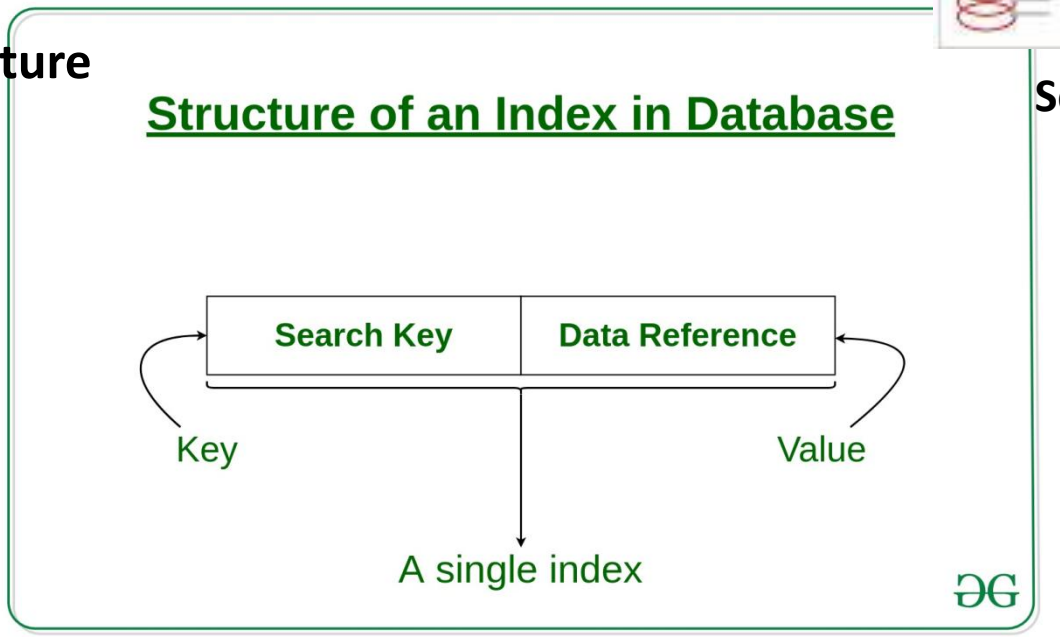
Indexing
and
M-way Search Trees

Memory Types & Data Organization

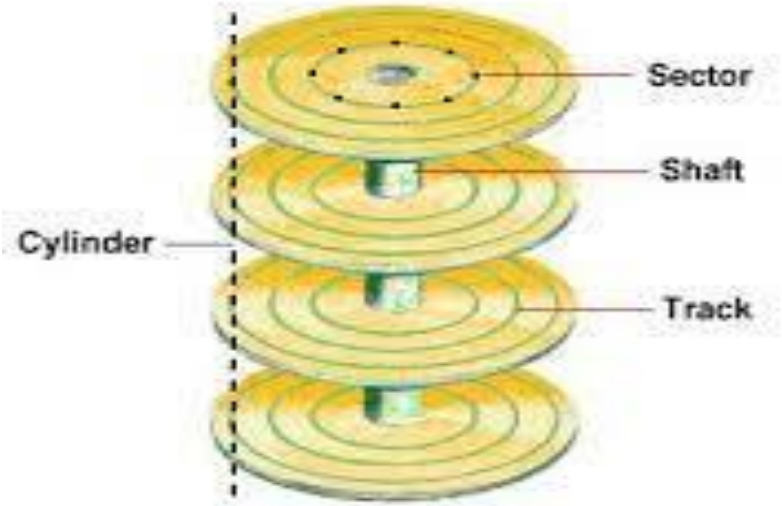
Main Memory (RAM)
Data Structure

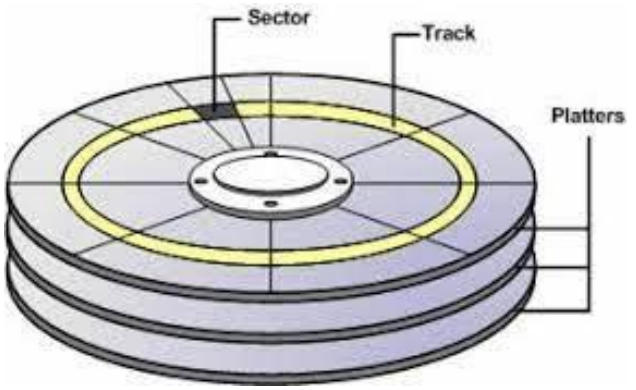


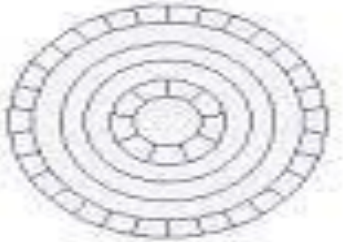
Secondary memory (Hard Disk)
DBMS

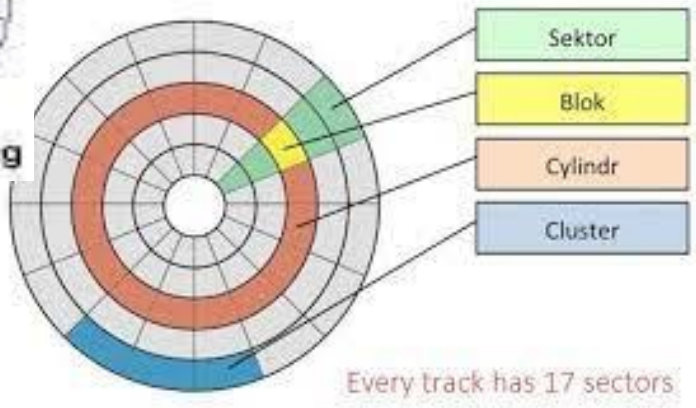


Disk Internal Structure









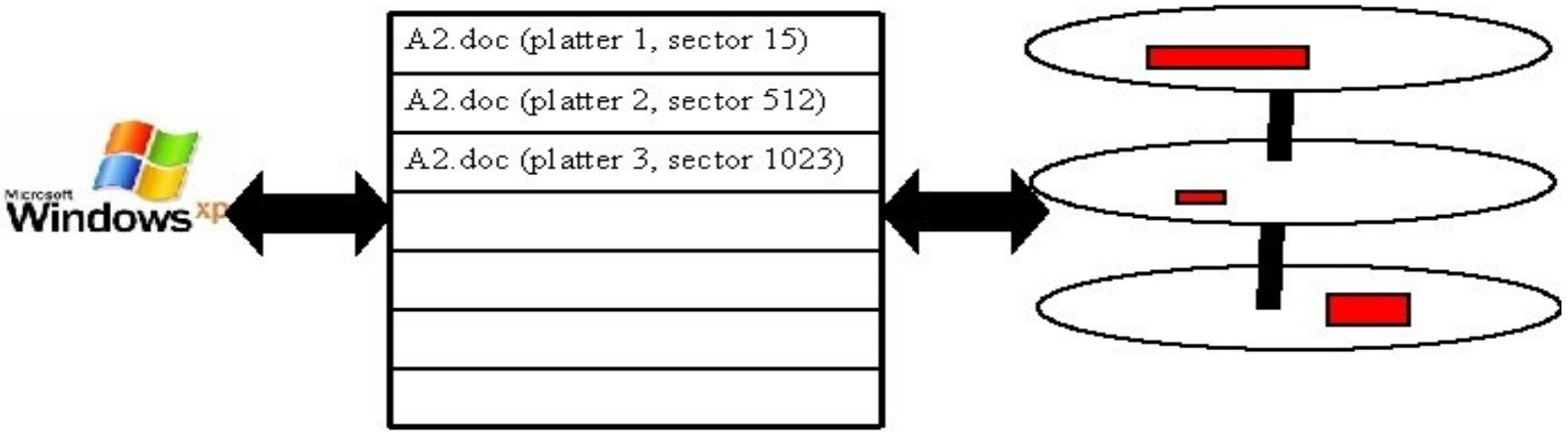
Each **platter** of a hard disk is divided into a number of concentric tracks.

Each track is divided into a number of sectors, each of which can store the same amount of data.

A sector is the smallest physical storage unit on the disk, and on most file systems it is fixed at 512 bytes in size.

Indexing Hard Drives

- To speed up the retrieval of information from storage, information is 'indexed'.



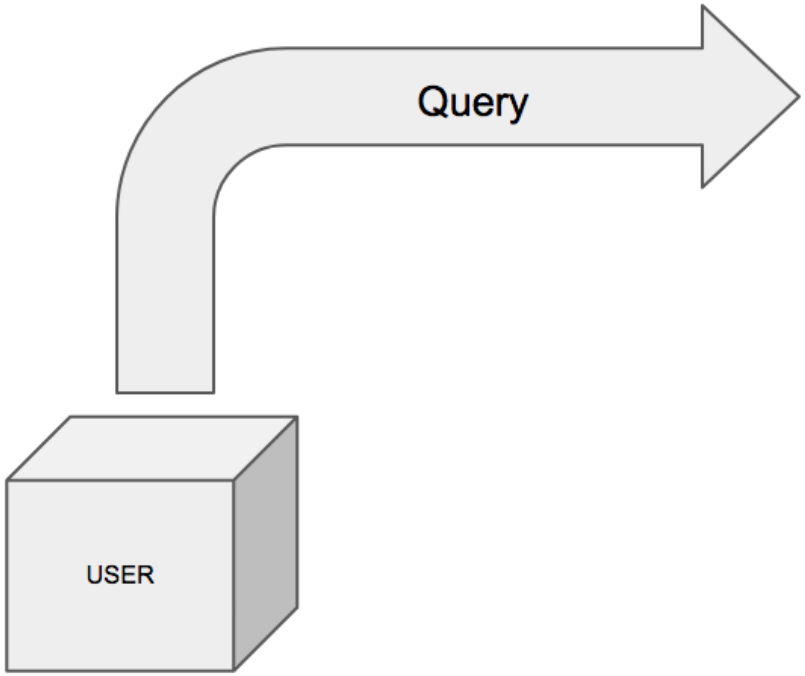
- When the operating system searches the hard drive it actually searches the index to the drive rather than the drive itself.

James' Tahn

What Does Indexing Do?

Indexing is the way to get an unordered table into an order that will maximize the query's efficiency while searching.

- Search record with **company_id =18**



Table

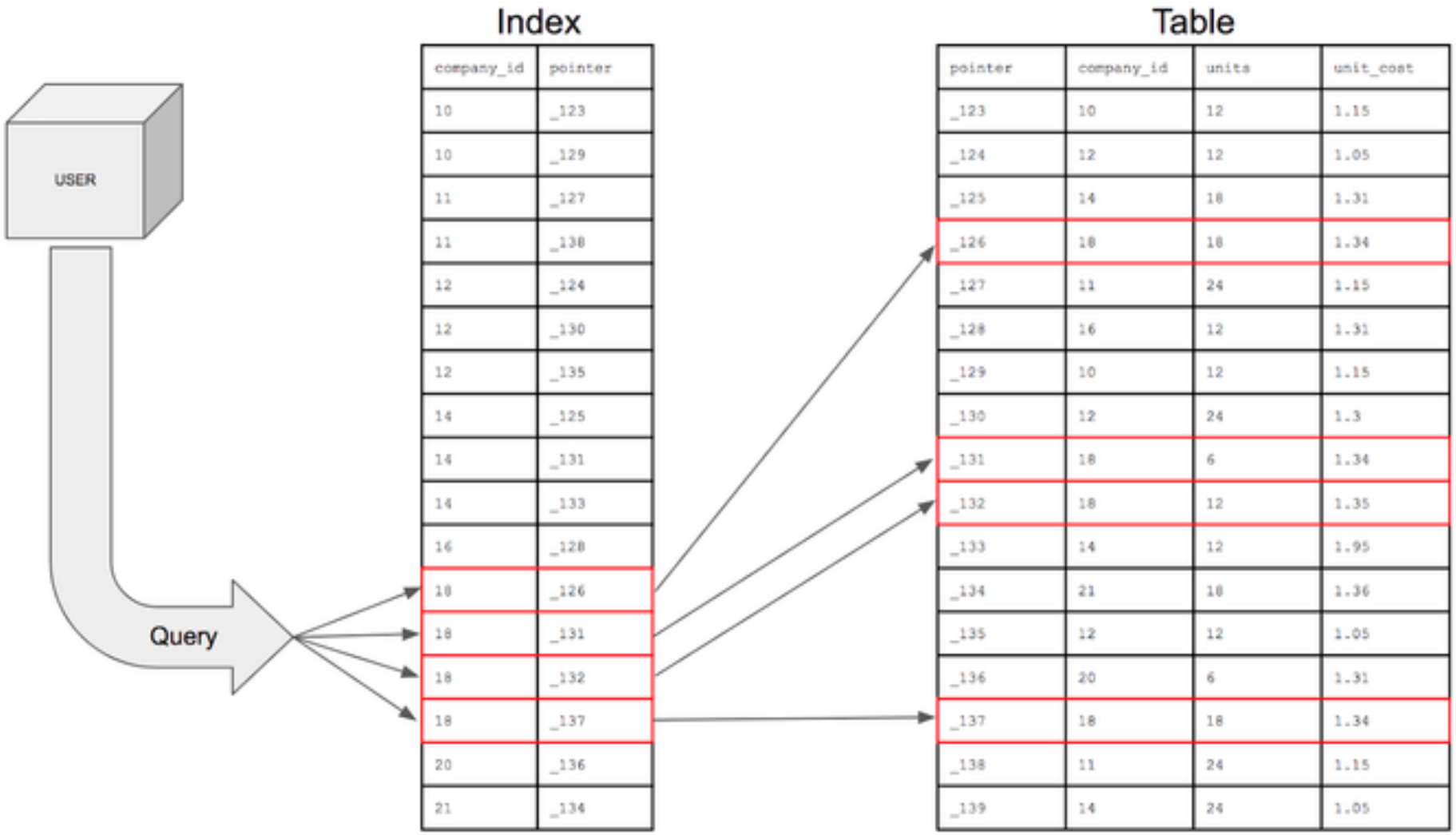
company_id	units	unit_cost
10	12	1.15
12	12	1.05
14	18	1.31
18	18	1.34
11	24	1.15
16	12	1.31
10	12	1.15
12	24	1.3
18	6	1.34
18	12	1.35
14	12	1.95
21	18	1.36
12	12	1.05
20	6	1.31
18	18	1.34
11	24	1.15
14	24	1.05

When a table is unindexed

- the queries will have to search through every row to find the rows matching the conditions.
- Looking through every single row is not very efficient.

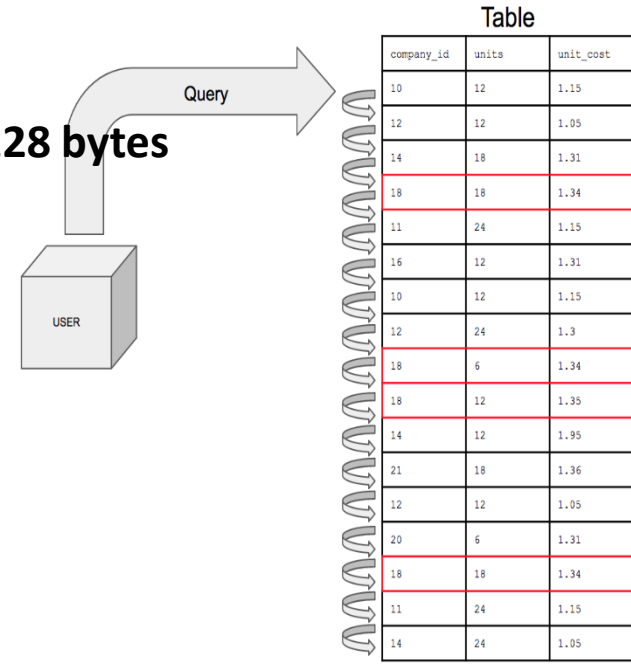
With an index on the `company_id` column, the table would be like this

When a table is Indexed



Understanding Indexing

A database of **200 records** stored where each **record requires 128 bytes**



Assuming each sector is 512 bytes

Therefore **1 sector represents 4 records**

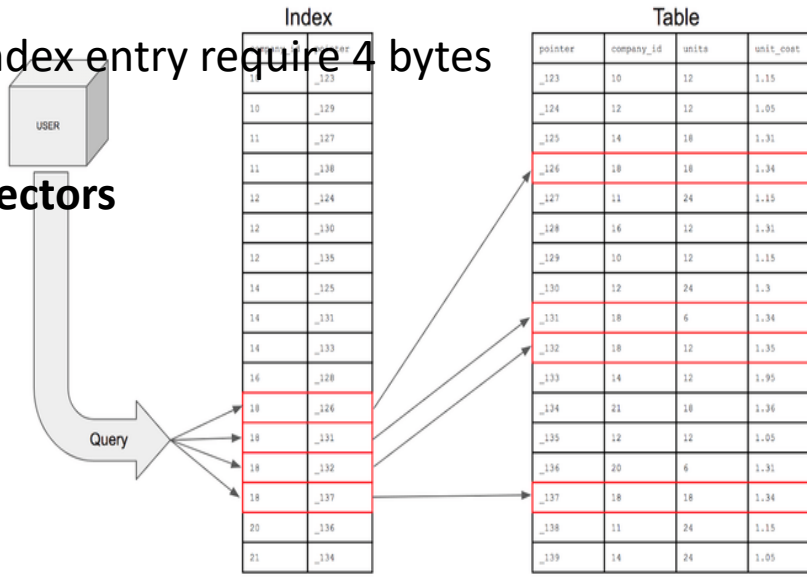
- So, entire database requires = 200 /4 = **50 sectors**

➢ **(thus 50 hard-disk read operations)**

With Indexing

- Indexing is done for each record and assume each index entry require 4 bytes
- Therefore for 200 record it need 200*4= **800 bytes**
- **Extra sector** required for index entry 800/512 = **2 sectors**

So total **2+ 1 = 3 reads are required to get the data**



Multilevel Indexing

(When data size is too large)

- Lets assume now to store **2000 records** , which requires 2000/4 **= 500 sectors**

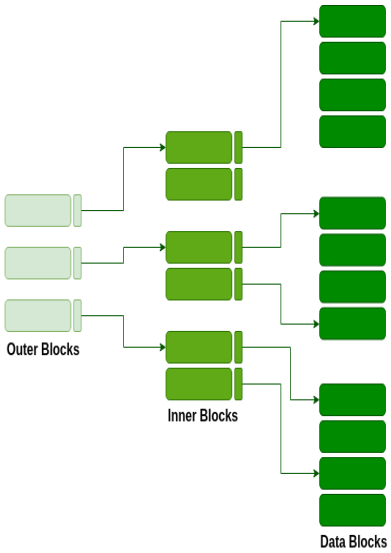
- Indexing will requires 2000*4 = 8000 => 8000/512 = 16 sectors

Total read operations **16 + 1 = 17 sector**

Efficient approach is to **make one more index for each sector**

- **Extra sector** required for index entry 16*4/512 = **1 sectors** (assume one entry for each sector)

So total **1 + 1 +1 = 3 reads are required to get the data**



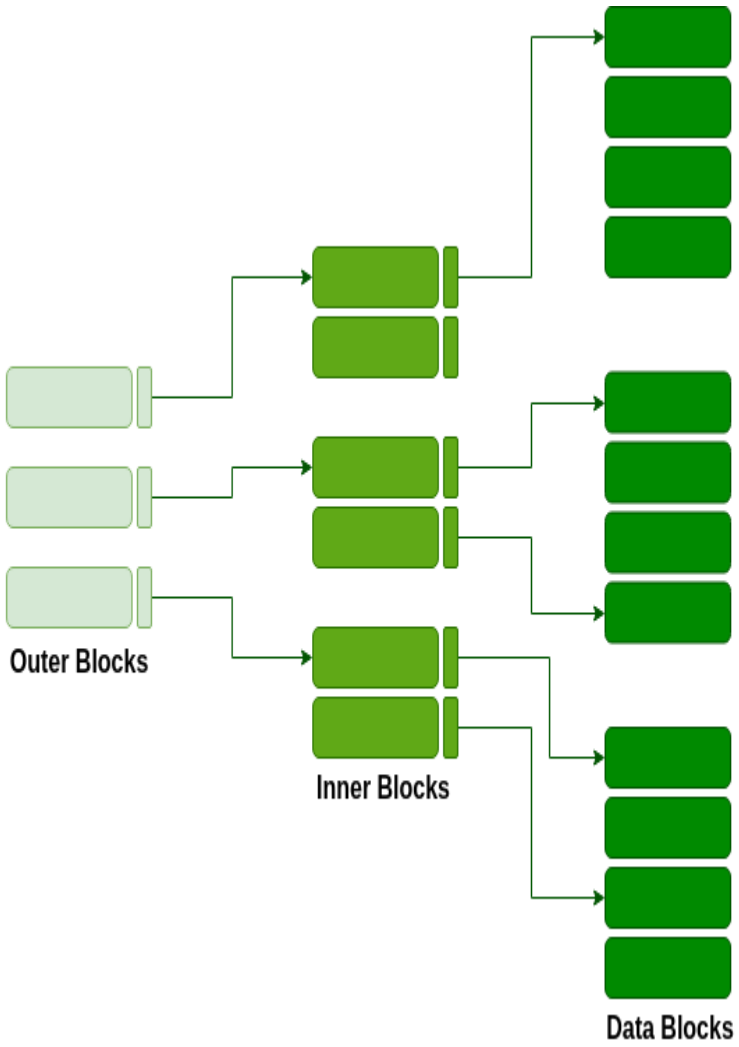
Multilevel Indexing

As the index is stored in the main memory, a single-level index might become too large a size to store with multiple disk accesses.

The multilevel indexing segregates the main block into various smaller blocks so that the same can stored in a single block.

The outer blocks are divided into inner blocks which in turn are pointed to the data blocks.

This can be easily stored in the main memory with fewer overheads.



Indexing

- Indexing adds a data structure with columns for the search conditions and a pointer
- The pointer is the address on the memory disk of the row with the rest of the information
- The index data structure is sorted to optimize query efficiency
- The query looks for the specific row in the index; the index refers to the pointer which will find the rest of the information.
- The index reduces the number of rows the query has to search

M-way Trees

A binary search tree has *one* value in each node and *two* subtrees.

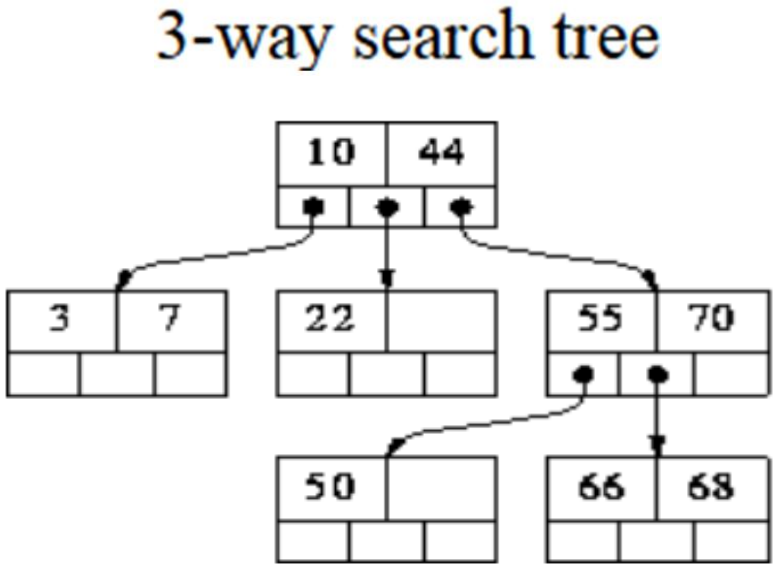
This notion easily generalizes to an M-way search tree, which has (M-1) values per node and M subtrees.

- M is called the *degree* of the tree. A binary search tree, therefore, has degree 2.
- it is not necessary for every node to contain exactly (M-1) values and have exactly M subtrees.
- In an M-way subtree a node can have anywhere from 1 to (M-1) values, and the number of (non-empty) subtrees can range from 0 (for a leaf) to 1+(the number of values).
- M is thus a *fixed upper limit* on how much data can be stored in a node.

The values in a node are stored in ascending order, V1 < V2 < ... Vk (k <= M-1) and the subtrees are placed between adjacent values, with one additional subtree at each end.
We can thus associate with each value a `left' and `right' subtree.

In practice, **M is usually very large**.

Each node corresponds to a physical block on disk, and M represents the maximum number of data items that can be stored in a single block.



M is maximized in order to speedup processing:
to **move from one node to another involves reading a block from disk**

- ✓ Reading from disk is very slow operation compared to moving around a data structure stored in memory.

Algorithm for searching for a value in an M-way search tree

Searching for value X and current node consisting of values V1...Vk

Four possible cases that can arise:

- If X < V1, recursively search for X in V1's left subtree.
- If X > Vk, recursively search for X in Vk's right subtree.
- If X=Vi, for some i, then we are done (X has been found).
- the only remaining possibility is that, for some i, **Vi < X < V(i+1).** In this case recursively search for X in the subtree that is in between Vi and V(i+1).

m-way trees have the following properties:

- Each node has 0 .. m subtrees
- A node with k < m subtrees, contains k-1 keys.
- The key values of the first subtree are all less than the key value.
- The data entries are ordered.
- All subtrees are m-way trees.

- As with binary search trees, inserting values in ascending order will result in a degenerate M-way search tree; i.e. a tree whose height is O(N) instead of O(logN).

- This is a problem because all the important operations are O(height), and it is our aim to make them O(logN).

- One solution to this problem is to *force* the tree to be height-balanced;

Construct a M-way Search Tree of **order 3** by inserting following nodes

20, 30, 40, 50, 10, 15, 25, 45, 90, 105, 95, 80, 35

Construct a M-way Search Tree of **order 4** by inserting following nodes

20, 30, 40, 50, 10, 15, 25, 45, 90, 105, 95, 80, 35